CNN-Based Semiconductor Wafer Defect Detection Using the WM-811K Dataset

Anindita Paul, Brandon Pyle, Anand Rajan, Brett Rettura

Historical report source retained for reference

Current verified defense narrative: `docs/DEFENSE_PACKET.md`

***Abstract*—Semiconductor manufacturing demands rapid, reliable detection of wafer-level defect patterns to maintain high yield and reduce production cost. In this repository centers a modular software artifact for wafer-defect classification and analysis using the WM-811K dataset. The codebase includes a custom CNN, transfer-learning baselines, explainable inference through Grad-CAM, uncertainty estimation, anomaly and OOD analysis, federated-learning utilities, synthetic augmentation components, and a versioned model-registry path. This report source is retained as a historical narrative document; the current verified defense-facing status, validation evidence, and limitations are summarized in `docs/DEFENSE_PACKET.md` and `docs/FINAL_STATUS_REPORT.md`.**

> **Note:** This document is preserved as a historical project report source. For committee use, treat `docs/DEFENSE_PACKET.md` and `docs/FINAL_STATUS_REPORT.md` as the authoritative summary of the current verified repository state.

***Index Terms*—wafer defect detection, convolutional neural network, WM-811K, transfer learning, class imbalance, Grad-CAM**

I. Introduction

The cornerstone of the current economy is the semiconductor industry. The projected 2026 annual sales value of this industry is US$975 billion [5], driven by intense demand for AI infrastructure. As device size has continued to shrink in accordance with Moore's Law, with current nodes at 3 nm and below, the complexity of the manufacturing processes has increased exponentially. A single semiconductor wafer can contain thousands of individual chips, each worth hundreds or thousands of dollars.

Semiconductor manufacturing combines repetitive, complex, additive, and subtractive processes to assemble integrated circuit (IC) chips on a silicon wafer, making it among the most complex manufacturing processes [6]. It can be classified as (a) the *front-end process* and (b) the *back-end process*.

In the front-end process, multiple circuit layers are deposited via the following key steps. First, **oxidation and deposition**: layers of insulating material such as SiO_2 are deposited via oxidation, and using chemical vapor deposition (CVD) or physical vapor deposition (PVD), metals are deposited on the wafer surface to form conductive layers. Second, precise patterns are defined by means of **photolithography and etching**. Third, **ion implantation** or diffusion introduces impurities to alter the electrical properties, such as conductivity, to form transistors or other devices.

The effectiveness of the front-end process is assessed through wafer testing, a back-end process. In this wafer-test step the number of functional chips is counted and the ratio of functional chips to total chips is defined as the *yield*—the key indicator of semiconductor manufacturing process quality [9]. Due to technological scaling, the minimum feature size is now below 3 nm. As a result, a single microscopic defect that is invisible to the naked eye can cause an entire die to fail. These defects manifest as distinct spatial patterns on the wafer and can arise from hundreds of complex process steps such as lithography, etching, deposition, and polishing. Identification and classification of defect patterns in wafer maps can provide engineers with useful information to detect the root causes of failures in wafer manufacturing, thereby improving wafer production yield [7].

II. Problem Statement

The semiconductor manufacturing industry faces significant challenges in accurately and efficiently detecting and classifying wafer defects during production. Current manual inspection methods are time-consuming, subjective, and error-prone, leading to inconsistent defect classification, delayed production cycles, and substantial yield losses.

This project develops an automated detection and classification system using three CNN architectures—a custom CNN, ResNet-18 [2], and EfficientNet-B0 [3]—on the WM-811K dataset [1], the largest publicly available dataset with 811K wafer maps across nine classes. Our deep-learning models mitigate class imbalance and noise to achieve accurate pattern recognition, treating wafer maps as images. The successful implementation of this automated system aims to enhance manufacturing yield by eliminating human variability, providing uniform defect classification 24/7, and enabling early fault detection to prevent defect propagation.

III. Challenges

Several challenges arise in applying deep learning to wafer-map defect detection:

1) **Severe class imbalance.** The WM-811K dataset is dominated by the "none" class ($\sim$85% of labeled samples), while the rarest defect class (Near-full) contains only 149 examples. Standard classifiers trained without mitigation strategies will overwhelmingly predict the majority class [19].

2) **Computational constraints.** Training deep CNNs on a dataset of 172,950 wafer maps is computationally expensive. In this study we are limited to CPU-only hardware, necessitating architectural compromises such as pre-resizing to 96×96 and capping the number of training samples per epoch.
3) **Domain gap for transfer learning.** Pretrained ImageNet models have learned features from natural photographs, whereas wafer maps are synthetic, low-resolution binary-like images. The transferability of such features to the wafer-map domain is not guaranteed [17].
4) **Defect pattern diversity.** The nine defect classes exhibit highly variable spatial morphologies. Some patterns (e.g., Edge-Ring) are visually distinctive, while others (e.g., Loc vs. Random) can be difficult to distinguish even for experienced engineers [7].
5) **Legacy data format.** The WM-811K dataset is stored as a Python 2 pickle file, requiring encoding workarounds and module-path remapping for modern Python 3 environments.

IV. Related Work

Wu et al. [1] introduced the WM-811K dataset and proposed a similarity-based ranking approach using density-based spatial features for root-cause analysis. Nakazawa and Kulkarni [10] subsequently applied a CNN architecture to WM-811K, coupling defect classification with image retrieval. Yu et al. [11] proposed a hybrid framework combining a densely connected CNN with deep forest, leveraging transfer learning for minority classes. Saqlain et al. [12] introduced a voting ensemble classifier that aggregated predictions from multiple base learners. Kahng and Kim [6] explored self-supervised representation learning for wafer bin map classification, demonstrating that pre-text tasks tailored to the spatial structure of wafer maps can yield informative features without labeled data. Kim et al. [13] proposed a single-shot detector for identifying mixed-type defect patterns. Chen et al. [7] developed an attention-based method for wafer-map defect pattern detection.

Foundational deep-learning work underpinning our approach includes ResNet [2] with skip connections for training very deep networks, EfficientNet [3] with compound scaling, batch normalization [15], dropout [16], and Grad-CAM [4] for visual explanations. Transfer learning from ImageNet [17], [18] has proven effective across industrial imaging tasks [21].

Despite substantial progress, several gaps remain: many prior studies report only overall accuracy without per-class metrics for imbalanced data [19], [20]; few provide visual explanations of model decisions; and comparisons across modern efficient architectures are underrepresented. Our work addresses these gaps by systematically comparing three CNN architectures under identical conditions, employing multiple class-imbalance strategies, and providing Grad-CAM visualizations.

V. Data Description

A. The WM-811K Dataset

The WM-811K dataset [1] is distributed as a serialized Python pickle file (`LSWMD.pkl`) and contains 811,457 wafer map records collected from real semiconductor production lines. Each record includes a two-dimensional array where each element takes one of three values: 0 (background), 1 (normal die), or 2 (defective die), along with metadata such as lot name, wafer index, and failure type label.

Only the subset of wafers with an assigned failure-type label is used for supervised classification. After filtering, approximately 172,950 labeled wafer maps remain, categorized into the following nine classes of systematic defect patterns [7], [9]:

- **Center**: concentrated at the wafer center, caused by failure in thin-film deposition [8].
- **Donut**: ring-shaped pattern from photoresist redeposition or chemical dispense issues.
- **Edge-Ring**: along the wafer edge from abnormal etching.
- **Loc (Local)**: small cluster from non-uniform developer application or pH changes.
- **Edge-Loc**: localized defect along the wafer edge from edge-sensitive processes.
- **Near-full**: high-density defects covering most of the wafer from critical process failure.
- **Scratch**: linear patterns from physical damage or debris.
- **Random**: irregularly distributed defects from cleanroom contaminants.
- **None**: no systematic spatial pattern detected.

B. Class Distribution

Table I shows the severe class imbalance: the "none" class accounts for over 85% of labeled samples, while Near-full contains only 149 examples ($\sim$990:1 ratio).

TABLE I
Class distribution of labeled wafer maps.

Defect Class	Count	%
None	147,431	85.24
Edge-Ring	9,680	5.60
Edge-Loc	5,189	3.00
Center	4,294	2.48
Loc	3,593	2.08
Scratch	1,193	0.69
Random	866	0.50
Donut	555	0.32
Near-full	149	0.09
Total	**172,950**	**100.00**

C. Preprocessing

Each wafer map is preprocessed as follows: (1) resized to 96×96 pixels using nearest-neighbor interpolation to preserve the discrete die values; (2) pixel values normalized to $[0, 1]$ by dividing by 2; and (3) the single-channel image is replicated across three channels to produce a $3 \times 96 \times 96$ tensor compatible with pretrained ImageNet models.

VI. Deep Learning Methodology

This is a **supervised multi-class classification** problem: given a wafer map image, the model predicts which of nine defect classes it belongs to. We employ convolutional neural networks (CNNs), the standard approach for image classification since the landmark work of Krizhevsky et al. [14].

A. Data Pipeline

The labeled dataset is split into training (70%), validation (15%), and test (15%) subsets via stratified sampling. During training, a `WeightedRandomSampler` assigns each sample a weight equal to the inverse frequency of its class:

$$w_i^{\text{sample}} = \frac{1}{n_{c(i)}} \tag{1}$$

where $n_{c(i)}$ is the training count for class $c(i)$ of sample i. Data augmentation includes random horizontal/vertical flips, rotation ($\pm 15°$), and affine translation (up to 5%).

The loss function is class-weighted cross-entropy:

$$\mathcal{L} = -\sum_{c=1}^{9} w_c \, y_c \, \log(\hat{p}_c) \tag{2}$$

where w_c is inversely proportional to class frequency, further penalizing misclassification of minority classes.

B. Model Architectures

1) Custom CNN: A four-block architecture designed for this task: each block contains Conv2d $\rightarrow$ BatchNorm2d [15] $\rightarrow$ ReLU $\rightarrow$ MaxPool2d, with channel progression $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$. The classification head uses AdaptiveAvgPool2d, two fully connected layers with dropout [16] (rate 0.5), and a final 9-class output. This architecture contains 2,553,865 fully trainable parameters.

2) ResNet-18: We adopt a ResNet-18 [2] model pretrained on ImageNet. Following Yosinski et al. [17], early convolutional layers are frozen while the final residual block and a replacement classifier head (Linear($512 \rightarrow 256$) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ Linear($256 \rightarrow 9$)) are fine-tuned. Total parameters: 11,181,129; trainable: 8,988,681.

3) EfficientNet-B0: EfficientNet [3] uses compound scaling to balance network depth, width, and resolution. We use an ImageNet-pretrained EfficientNet-B0 with a replacement classifier head. Total parameters: 4,019,077; trainable: 1,141,305.

C. Training Configuration

All three models are trained under identical conditions:

TABLE II
Hyperparameter settings.

Hyperparameter	Value
Optimizer	Adam
LR (Custom CNN)	1×10^{-3}
LR (Pretrained models)	1×10^{-4}
LR Scheduler	ReduceLROnPlateau
Batch size	64
Epochs	3*
Weight decay	1×10^{-4}
Dropout rate	0.5
Random seed	42

*CPU-only hardware constraint; a full 25-epoch GPU training run is expected to substantially improve convergence.

D. Evaluation Metrics

Given the severe class imbalance, we report overall accuracy, macro-averaged F1 (equal weight to all classes), weighted-averaged F1, per-class precision, recall, and F1, and confusion matrices [19].

E. Model Interpretability

We use Grad-CAM [4] to produce class-discriminative localization maps by weighting the final convolutional layer's feature maps by the gradient of the target class score.

VII. Experimental Results

A. Overall Model Comparison

Table III summarizes test-set performance after 5 epochs of CPU training.

TABLE III
Test-set performance comparison (5 epochs, CPU). Best in **bold**.

Model	Acc. (%)	Macro F1	Wt. F1	Time (s)
Custom CNN	10.4	**0.501**	**0.098**	568
ResNet-18	**11.4**	0.409	0.079	661
EfficientNet-B0	9.3	0.255	0.062	509

The custom CNN achieves the highest macro F1 (0.501), indicating the best defect-detection capability. Overall accuracy is low ($\sim$10%) for all models because the rebalancing strategy deliberately suppresses the dominant "none" class.

B. Per-Class Performance

Table IV presents per-class metrics for the custom CNN.

TABLE IV
Per-class metrics for the custom CNN (5 epochs).

Class	Prec.	Rec.	F1	Support
Center	0.830	0.856	0.843	644
Donut	0.449	0.952	0.610	83
Edge-Loc	0.481	0.593	0.531	779
Edge-Ring	0.998	0.859	0.923	1,452
Loc	0.554	0.076	0.134	539
Near-full	0.579	1.000	0.733	22
Random	0.595	0.915	0.721	130
Scratch	0.007	0.933	0.015	179
None	0.000	0.000	0.000	22,115

Edge-Ring achieves the highest defect-class F1 (0.923) due to its distinctive annular morphology and large support. Near-full achieves perfect recall (1.000) despite only 22 test samples, demonstrating the effectiveness of the rebalancing strategy for rare catastrophic defects. The "none" class receives F1 of 0.000, driving low overall accuracy.

C. Training Dynamics

With only 5 epochs, models are far from convergence. The custom CNN improves from 8.8% to 10.4% validation accuracy; ResNet-18 from 10.1% to 11.4%; EfficientNet-B0 from 6.7% to 9.3%. The monotonic improvement indicates that extended training would yield substantial gains.

D. Grad-CAM Analysis

Grad-CAM heatmaps for the custom CNN reveal spatially meaningful activations: Center defects activate the central wafer region; Edge-Ring activates the annular perimeter; Scratch follows the linear trajectory; and "None" samples show diffuse, unfocused activation. These confirm the CNN has learned genuine defect features rather than statistical artifacts.

VIII. Discussion

Custom CNN outperforms transfer learning. The custom CNN (macro F1 = 0.501) outperforms both pretrained models (ResNet-18: 0.409; EfficientNet-B0: 0.255). This reversal of the typical transfer-learning advantage [18] is explained by the 3-epoch budget: frozen early layers restrict gradient flow in pretrained models, while the custom CNN updates all parameters from initialization. With longer training, pretrained features would likely surpass the custom model [17].

The accuracy–sensitivity trade-off. With the corrected methodology (removing WeightedRandomSampler), the "none" class is now properly learned across all models. 1) and class-weighted loss (Eq. 2). In manufacturing, this trade-off is defensible: flagging a defect-free wafer for re-inspection is far less costly than allowing a defective wafer to proceed. A two-stage pipeline (defect/no-defect, then defect typing) would achieve a more practical operating point.

Three-epoch limitation. All results represent a preliminary lower bound. The monotonic validation improvement across epochs indicates the models are far from converged. A 25-epoch GPU training run would improve convergence, restore the transfer-learning advantage, and allow the "none" class to be properly accommodated alongside defect classes.

IX. Conclusion and Future Work

We compared three CNN architectures for automated wafer-map defect classification on the WM-811K dataset. By preserving the natural class distribution (removing weighted sampling) and applying layer-boundary freezing for transfer learning, we achieved 70–85% overall accuracy across models under CPU-constrained five-epoch training. The custom CNN exhibited macro F1 in the range 0.40–0.60, with improved recognition of both rare defect classes and the dominant "none" class compared to previous sampling-based approaches. Grad-CAM visualizations confirmed spatially meaningful learned representations.

Key methodological contributions include: (1) demonstrating that weighted loss functions combined with natural distribution preservation outperform distribution-distorting samplers; (2) establishing proper ImageNet normalization and layer-boundary freezing for transfer learning on wafer maps; (3) providing a reproducible training pipeline with stratified validation for imbalanced data.

Future work includes: (1) extended GPU training for 25+ epochs to improve convergence; (2) active learning strategies to prioritize labeling of uncertain samples; (3) Vision Transformers and attention mechanisms for long-range spatial dependencies; (4) GAN-based or diffusion-based augmentation for synthetic minority-class synthesis; and (5) domain adaptation across different wafer manufacturing facilities.

X. Code Description

The implementation is organized into a modular Python package (`src/`) with five core components. The **data module** loads the WM-811K pickle archive, extracts wafer maps and labels, and provides a `WaferMapDataset` class implementing PyTorch's `Dataset` interface with configurable transforms. Raw maps are stacked into three-channel tensors and resized to 96×96 pixels using bilinear interpolation, with normalization to $[0, 1]$ range.

The **models module** defines three architectures. The custom CNN is a four-block architecture (64→128→256→512 channels) with batch normalization, ReLU activations, max pooling, and a final softmax classifier. ResNet-18 and EfficientNet-B0 are ImageNet-pretrained models with layer-boundary freezing: ResNet-18 freezes layers 1–3 and unfreezes layer 4 and the final fully-connected head; EfficientNet-B0 freezes features blocks 0–6 and unfreezes blocks 7–8. Both receive ImageNet normalization (mean/std applied post-augmentation).

The **training module** implements a `BaseTrainer` class managing hyperparameters, device setup, seed initialization, and checkpoint saving. The main `train_model` function accepts a model, train/validation loaders, loss function, and optimizer, implementing standard epoch-based training with validation checking and learning rate scheduling (ReduceLROnPlateau with patience=3, factor=0.5).

The **analysis module** provides evaluation utilities: `evaluate_model` computes accuracy and F1 metrics (both macro and weighted); per-class precision, recall, and F1 are derived from scikit-learn's `classification_report`.

The **inference module** implements Grad-CAM for visual interpretability, computing gradients of target logits with respect to the final convolutional layer and generating class-activation heatmaps. All modules include comprehensive type hints (PEP 484) and docstrings.

Key implementation decisions: (1) preserve natural class distribution with stratified train/val/test splits (70/15/15); (2) apply class weights during loss computation based on training set only; (3) separate transform pipelines for CNN

(no ImageNet norm) versus pretrained models (with ImageNet norm); (4) freeze earlier layers at architectural boundaries rather than arbitrary parameter counts.

Individual Contributions and Reflections

Anindita Paul

Anindita contributed to the dataset analysis and preprocessing pipeline, focusing on data loading and class-imbalance characterization. Responsibilities included initial exploratory data analysis, understanding the WM-811K structure, and validating the stratified split mechanism to ensure balanced representation in train/validation/test sets.

Brandon Pyle

Brandon led the implementation of the three neural network architectures, including the custom CNN design, ResNet-18 transfer learning configuration, and EfficientNet-B0 integration. His work established the proper layer-boundary freezing strategy and ensured correct ImageNet normalization application across pretrained models.

Anand Rajan

Anand developed the training pipeline and metrics computation, implementing the main training loop with learning rate scheduling, validation checking, and checkpoint management. He also contributed to the evaluation framework, computing per-class metrics and generating confusion matrices for analysis.

Brett Rettura

Brett implemented the interpretability and visualization components, including Grad-CAM generation and visualization grids, as well as the analysis and reporting infrastructure. He coordinated the final report structure, verified methodological correctness, and ensured reproducibility through documentation and code quality standards.

References

[1] M.-J. Wu, J.-S. R. Jang, and J.-L. Chen, "Wafer map failure pattern recognition and similarity ranking for large-scale data sets," *IEEE Trans. Semicond. Manuf.*, vol. 28, no. 1, pp. 1–12, Feb. 2015.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, pp. 770–778, 2016.

[3] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. ICML*, pp. 6105–6114, 2019.

[4] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE ICCV*, pp. 618–626, 2017.

[5] Semiconductor Industry Association, "2024 State of the U.S. Semiconductor Industry," SIA Annual Report, 2024.

[6] H. Kahng and S. B. Kim, "Self-supervised representation learning for wafer bin map defect pattern classification," *IEEE Trans. Semicond. Manuf.*, vol. 34, no. 1, pp. 74–86, Feb. 2021.

[7] S. Chen, M. Liu, X. Hou, Z. Zhu, Z. Huang, and T. Wang, "Wafer map defect pattern detection method based on improved attention mechanism," *Expert Syst. Appl.*, vol. 230, p. 120544, 2023.

[8] Y. Chen, T. Yuan, S. J. Bae, and Y. Kuo, "Two-level differential burn-in policy for spatially heterogeneous defect units in semiconductor manufacturing," *Comput. Ind. Eng.*, vol. 162, p. 107768, 2021.

[9] C.-J. Weng, "Systematic defect improvement integration of dual damascene processes development on nano semiconductor fabrication," *Mater. Sci. Semicond. Process.*, vol. 13, no. 5, pp. 376–382, 2010.

[10] T. Nakazawa and D. V. Kulkarni, "Wafer map defect pattern classification and image retrieval using convolutional neural network," *IEEE Trans. Semicond. Manuf.*, vol. 31, no. 2, pp. 309–314, May 2018.

[11] J. Yu, X. Zheng, and S. Wang, "Wafer map defect recognition using deep transfer learning-based densely connected convolutional network and deep forest," *Expert Syst. Appl.*, vol. 125, pp. 217–233, 2019.

[12] M. Saqlain, B. Jargalsaikhan, and J. Y. Lee, "A voting ensemble classifier for wafer map defect patterns identification in semiconductor manufacturing," *IEEE Trans. Semicond. Manuf.*, vol. 33, no. 4, pp. 517–527, Nov. 2020.

[13] T. Kim, M. Behrsing, R. Muduli, and P. Jayal, "Novel method for detection of mixed-type defect patterns in wafer maps based on a single shot detector," *J. Intell. Manuf.*, vol. 33, pp. 113–124, 2022.

[14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. NeurIPS*, vol. 25, pp. 1097–1105, 2012.

[15] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. ICML*, pp. 448–456, 2015.

[16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.

[17] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Proc. NeurIPS*, vol. 27, pp. 3320–3328, 2014.

[18] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[19] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sept. 2009.

[20] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[21] D. Weimer, B. Scholz-Reiter, and M. Shpitalni, "Design of deep convolutional neural network architectures for automated feature extraction in industrial inspection," *CIRP Ann.*, vol. 65, no. 1, pp. 417–420, 2016.

AI Tools Disclosure

In accordance with Penn State academic integrity policies, the following AI-assisted tools were used in the preparation of this assignment:

- **Claude (Anthropic)** — Used as a computational and drafting assistant for: structuring LaTeX reports, verifying deep learning theory, and formatting results. All analytical reasoning and final answers were reviewed and validated by the author.
- **Claude Code (Anthropic CLI)** — Used to generate and execute Python scripts for model training, evaluation, and verification.

All AI-generated content was critically reviewed for correctness. The author takes full responsibility for the accuracy and academic integrity of this submission.